

Assignment - 3

NLP - CS60075

Spring Semester 2026, IIT Kharagpur

Implementing BERT from Scratch for Fake News Classification

Assignment Deadline: 15 April 2026, 11:59 pm IST

General Guidelines:

- This assignment is to be done by the pre-decided student groups. You should not copy any code from other groups or from any web source. We will use standard plagiarism detection tools on the submissions. Plagiarized codes will be awarded zero for the assignment, and we will not differentiate between who copied from whom.
- In this assignment, you are allowed to use Python libraries like PyTorch, Spacy, NLTK, numpy, pandas, and matplotlib, etc. for performing different sub-tasks. Your code should follow an object-oriented approach as described in the tasks.
- You can utilize free GPU services to run your experiments, e.g., Google Colab or Kaggle.
- Follow the submission instructions given at the end. Solutions should be uploaded via the CSE Moodle website (see course website for details).
- You should organize your code into different functions and add explanatory comments. Make sure your code can be easily read. Codes that cannot be understood will be penalized.
- *Any queries about this assignment should be mailed to TA, Amogh Joshi (amoghjoshi500@gmail.com)*

Overview

In this assignment, you will implement an encoder-only Transformer (BERT) from scratch to gain a deep understanding of its core components.

You will work with the [Fakeddit](#) dataset, which contains the posts from reddit and the comments on the respective posts. Your task is to classify whether the post is fake or not.

The report, which needs to be submitted with this assignment, carries **20 MARKS**.

Tasks

Task 1: Implement BERT class from scratch

(30 MARKS)

Objective: Implement the essential components of a BERT model from scratch using PyTorch, and test it on a sample sentence. The main goal of this task is to understand the data flow and the shapes of the embeddings. I've provided the references to guide your understanding. Please use them for reference only, not as-is.

Requirements:

1. **Implement the following components in PyTorch:** Consider number of heads = 8, embedding dimension = 512, number of layers = 2
 - Multi-Head Self-Attention
 - Feed-Forward Layer
 - Transformer Encoder Layer
 - BertModel (stack of encoders with token embeddings, positional embeddings, pooler, and classifier head)
2. **Tokenizer:** Use [bert-base-uncased](#) from Hugging Face to tokenize inputs.
3. **Testing with a Sample Sentence:**
 - Consider any sentence of your choice with **more than 10 words**.
 - Tokenize the sentence using the tokenizer.
 - Pass the tokenized input through your implemented BERT model and for the segmentation information (if required) pass 0 for all the tokens.
4. **Report:**
 - Record and include the **shapes of embeddings** at each stage of the model:
 - Token embeddings
 - Positional embeddings
 - Output of Multi-Head Attention
 - Output of Feed-Forward layer
 - Output of Transformer Encoder Layer
 - Output probabilities from the classifier
 - Shape of each parameter.

Task 2: Data Preprocessing

(10 MARKS)

Prepare the [Fakeddit](#) dataset for fake news classification by filtering, cleaning, and organizing the data.

Steps :

1. Navigate to the folder *all samples (also includes non multimodal)*. Load and combine the files - *all_train.tsv*, *all_test_public.tsv* and *all_validate.tsv*. Retain only the columns 'clean title' for the post content, '2_way_label' for the classification and 'id' for identifying the post.
2. Balance the Dataset:
 - a. Create a balanced dataset by randomly sampling equal numbers (consider at least 2500 each) of fake and non-fake posts.
3. Train-Test Split:
 - a. Split the filtered dataset into 80% training and 20% testing.
 - b. Make sure that both sets have balanced classes.
4. Report Statistics:
 - a. Number of posts in the original dataset
 - b. Number of posts which are fake and non-fake in the balanced dataset you built.
 - c. Distribution of posts in train and test sets

Task 3: Fine tune the model and Report Metrics

(20 MARKS)

Steps :

1. For this part, there is **no need** to use the **class you implemented in Task 1**.
2. Use `BertForSequenceClassification` from transformers and load the model `bert-base-uncased` to finetune the model.
3. If you are using only a subset of the input tokens, specify the exact values in your report and clearly describe the method used for truncating the input.
4. Set Hyperparameters and Train:
 - a. Define hyperparameters (epochs, batch size, learning rate, optimizer).
 - b. Finetune the model using cross-entropy loss on the training set.
5. Evaluate Performance:
 - a. Test the model on the test set and compute Accuracy, Precision, Recall, F1-score, confusion matrix.
6. Report Results:
 - a. Present a metrics table for the test set.
 - b. Document all hyperparameters used.

Task 4: Use Integrated Gradients for token attribution

(20 MARKS)

Objective:

Explain the predictions made by the fine-tuned BERT model using Integrated Gradients (IG) to identify which input tokens contributed most to the final classification.

Steps:

1. Load the Fine-Tuned Model:
 - a. Load the `BertForSequenceClassification` model fine-tuned in Task 3.
 - b. Set the model to evaluation mode.
2. Prepare Input text:
 - a. Take 3 examples for both fake and not-fake labels from the test set. Mention the sentences used in the report as well.
 - b. Retain `input_ids`, `attention_mask`, and `token_type_ids` (if used)
3. Defining the forward function:
 - a. Define a function that would take the embeddings of the input, pass through the model and return logits for the predicted class.
 - b. This function will be used by IG for attribution computation.
4. Integrated Gradients:
 - a. Use `captum` library for defining and using IG.
 - b. Use the `LayerIntegratedGradients` object with the defined forward function and choose the layer as the `embedding_layer`.
 - c. Use **all-zero embeddings** for the baseline inputs. Also clearly specify the target class index for which attributions are computed.

- d. Apply Layer Integrated Gradients to compute attributions for each token embedding. Also return the convergence delta value and adjust the n_steps hyperparameter until the convergence delta has an acceptable value.
- e. Aggregate attribution values across the embedding dimension to obtain a **single importance score per token**. Normalize the attribution scores if required for interpretation.
- f. Map attributions to tokens:
 - i. Convert token IDs back to tokens using the tokenizer.
 - ii. Align attribution scores with their corresponding tokens.
 - iii. Exclude special tokens such as [CLS], [SEP], and [PAD] from interpretation
 - iv. Print the top-5 tokens for each input and also mention them in the report.

Submission guidelines:

- Submit a single .zip file which contains **only two files**, .ipynb file and the report in pdf format.
- Name the zip file as Roll_Number_Assignment_3.zip
- Make sure your code in the .ipynb file has all the cell outputs.
- Make only **one submission per team**. The submission should be directly through Moodle. Submission through external media such as Google Drive links is NOT allowed.

References

- <https://medium.com/@whyamit101/fine-tuning-bert-a-practical-guide-b5c94efb3d4d>
- <https://medium.com/data-and-beyond/complete-guide-to-building-bert-model-from-scratch-3e6562228891>
- Integrated Gradients: https://captum.ai/docs/extension/integrated_gradients
- IG paper: <https://arxiv.org/abs/1703.01365>